

OOP Homework

1. Design Choices

The dungeon adventure game is designed to be a text-based, turn-based RPG where the player navigates through a series of interconnected rooms. Each room presents unique challenges, such as enemies, treasures, weapons, or puzzles. The game is structured around the following design principles:

- **Modularity:** The game is divided into classes (Enemy, Room, Player, Dungeon, etc.) to encapsulate specific functionality. This makes the code easier to maintain and extend.
- **Data Structures:**

The game uses two primary data structures:

 - **Queue:** Implemented in the EnemyQueue class to manage enemies in each room. This ensures that enemies are encountered in the order they are added.
 - **Stack:** Implemented in the RoomStack class to track the player's movement through the dungeon. This allows the player to backtrack to previous rooms.
 - **Dynamic array:** Implemented in the Player class for the inventory. This allows the player to store items that they have picked up through out the game.
- **Randomization:** Enemies and treasures are randomized to ensure replayability. For example, enemy types and stats are generated dynamically using rand().
- **Player Progression:** The player gains strength by collecting weapons and treasures, which increase their attack power and health. This creates a sense of progression and reward.

2. Implementation of Required Data Structures

a. Queue (EnemyQueue)

The EnemyQueue class is implemented using a linked list to manage enemies in each room. Key features:

- **Enqueue:** Adds an enemy to the end of the queue.
- **Dequeue:** Removes an enemy from the front of the queue.
- **Front:** Retrieves the enemy at the front of the queue without removing it.

This structure ensures that enemies are encountered in the order they are added, simulating a first-come, first-served battle system.

b. Stack (RoomStack)

The RoomStack class is implemented using a linked list to track the player's movement through the dungeon. Key features:

- **Push:** Adds a room to the top of the stack when the player moves forward.
- **Pop:** Removes a room from the top of the stack when the player backtracks.
- **IsEmpty:** Checks if the stack is empty.

This structure allows the player to backtrack to previous rooms, providing flexibility in navigation.

3. Challenges Faced

a. Managing Enemy Battles

One of the main challenges was implementing the enemy battle system. Each room can have multiple enemies, and the player must defeat them in sequence. The EnemyQueue was used to manage this, but ensuring that the player could retreat from battles without breaking the game logic required careful handling of the queue and player input.

b. Backtracking Mechanism

Implementing the backtracking system using a stack was challenging because it required tracking the player's movement history while ensuring that the stack was updated correctly when the player moved forward or backward. Additionally, the game had to handle edge cases, such as when the player tried to backtrack from the starting room.

c. Randomization and Balance

Balancing the game's difficulty was another challenge. Randomizing enemy stats and room contents while ensuring the game remained fair and enjoyable required extensive testing. For example, the boss enemy (Dragon) had to be significantly stronger than regular enemies but not so strong that the game became unwinnable.

d. Player Inventory Management

The player's inventory system had to dynamically resize to accommodate collected items. This was implemented using a dynamically allocated array, but managing memory allocation and deallocation added complexity.

4. Lessons Learned

- **Data Structures:** Using queues and stacks simplified the implementation of enemy management and room navigation. These structures were well-suited to the game's requirements.
- **Modularity:** Breaking the game into classes made it easier to debug and extend. For example, adding new room types or enemy behaviors would require minimal changes to the existing code.
- **Testing:** Extensive testing was necessary to ensure the game was balanced and free of bugs. Playtesting revealed issues with enemy difficulty and inventory management that were not apparent during development.

This design and implementation demonstrate how data structures and object-oriented programming can be used to create a functional and engaging game. The challenges faced during development provided valuable insights into game design and programming.